

Course Name: Computer Graphics Fundamentals

Project Title: Baikonur Rocket Launch

Team Members: Akbope Bakytkeldy, SE2426
Moldir Omar, SE2426
Ayaulym Talgat, SE2427
Talant Altaikhan, SE2425

Instructor: Nazira Shaimerdenova

Date of Submission: To be confirmed by the team

Table of Contents

1 Introduction 3

2 Literature Review 4

2.1 Particle and fire representations 4

2.2 Turbulence and rendering cost 5

2.3 Materials and atmospheric appearance 6

3 Project Description 7

4 Technical Implementation 8

4.1 Tools and technologies used 8

4.2 Development process 8

4.3 Algorithms and mathematical model 9

5 Graphics and Rendering 10

5.1 Scene construction and lighting 10

5.2 Shading and texture sampling 11

5.3 Particle animation and separation 12

6 Testing and Evaluation 13

7 Conclusion and Future Work 14

8 References 15

9 AI Use Note 18

Appendix A Course connections and defense 19

Deliverables

The accompanying source package contains the C++ application, GLSL shaders, bundled dependencies, tests, build instructions and evidence logs. The presentation contains ten slides. The captioned demonstration runs for 3 minutes 20 seconds at 1920 by 1080 pixels.

1 Introduction

Baikonur Rocket Launch is an interactive computer graphics application that presents a countdown, engine ignition, ascent and booster separation from a stylized launch complex. The project combines the course pipeline into one native C++ application: indexed geometry, programmable shaders, matrix transformations, lighting, texture sampling and interactive viewing.

A rocket launch makes the relationship between these techniques visible. The gantry and ground provide stable spatial references. The rocket body and four boosters demonstrate hierarchical transformations. Ignition introduces a moving point light and two different particle materials. A chase camera connects the flight path to the viewing transformation, while an altitude-dependent sky shows how a simulation parameter can drive a fragment shader.

The application is intended for classroom demonstration and explanation. The rocket and site are original procedural approximations inspired by the selected Baikonur topic. Their dimensions, trajectory and timing are compressed to make the complete sequence easy to inspect. The result is a graphics study rather than an aerospace simulator or an architectural reconstruction of a specific launch pad.

1.1 Expected outcome

The principal outcome is a repeatable, interactive scene that exposes its graphics features through keyboard, mouse and on-screen controls. Users can inspect the launch site, switch lighting, view wireframe geometry, launch the rocket, follow its ascent, separate boosters, pause the mission and reset it. A visible FPS counter and exported frame-time logs support performance evaluation.

The implementation includes all three topic-specific stretch features: booster detachment with independent motion, ignition camera shake, and stars that become visible at altitude. The report separates these implemented visual effects from advanced research techniques discussed as alternatives. It also records which validation tasks remain specific to the final teaching-assistant environment.

1.2 Relation to the course

The implementation follows the concepts in the supplied lectures on transformations, viewing, shading, texturing, hierarchical models and procedural methods [16]. The final assessment requires OpenGL 3.3 or newer in core profile. Consequently, the project uses GLFW and a GLAD loader with explicit matrices and GLSL rather than the historical fixed-function examples that appear in some lecture slides.

2 Literature Review

2.1 Particle and fire representations

The review compares fifteen journal articles [1]-[15] from ACM Transactions on Graphics and Computer Graphics Forum. It focuses on decisions relevant to a small interactive launch scene: visual control, simulation cost, material representation and atmospheric appearance. The cited papers establish alternatives and limitations; the project does not claim to implement every reviewed method.

Reeves [1] represents fuzzy phenomena through many particles whose birth, motion, appearance and death vary over time. This representation is a direct conceptual fit for exhaust and smoke because a cloud has no single stable polygonal boundary. The project adopts a finite particle lifecycle with randomized initial velocity, size and lifetime. Its camera-facing quads differ from a full participating-volume renderer but keep the lifecycle visible and inexpensive.

Nguyen, Fedkiw and Jensen [5] model fuel and hot gaseous products using fluid dynamics and render the resulting fire through a physically based treatment. This connects flame motion, buoyancy and radiance more closely than independent particles can. Such a system would improve combustion realism, but its field representation and numerical solution would substantially expand the scope of a course project. The implemented flame therefore uses age-based color, downward velocity and additive blending.

Lamorlette and Foster [6] emphasize artistic control of flame behavior through structural and stochastic mechanisms. Their approach shows that controllable motion is itself an important production objective. In this project, explicit emission rate, lifetime, speed and color parameters serve a similar design objective at a much simpler level. No spline-based flame skeleton or production flame system from that paper is implemented.

Rasmussen and colleagues [7] combine fluid-generated motion with additional turbulent detail to produce large-scale smoke. Their work explains why a convincing plume needs structure at several spatial scales. The project instead uses expanding billboards with outward drift near the pad. This is sufficient to distinguish smoke from fire, but it cannot reproduce coherent rolling vortices or light attenuation through a dense volume.

Design implication

These papers support a deliberate scope decision: maintain a clear and bounded particle lifecycle first, then identify fluid simulation and richer turbulent structure as future work. The visible effect must remain responsive to launch, pause and reset commands. This favors a compact CPU pool and GPU instancing over a large volumetric simulation.

2.2 Turbulence and rendering cost

Losasso, Gibou and Fedkiw [8] use an octree to concentrate simulation resolution where detail matters. This reduces the waste of uniformly fine grids but introduces adaptive data structures and pressure-solver complexity. The launch project has a different bottleneck and scale: a small, bounded particle set. It therefore shares geometry and batches particle rendering rather than allocating a fluid grid.

Selle, Rasmussen and Fedkiw [9] combine vortex particles with an Eulerian representation to recover turbulent detail that can be lost in grid-based advection. Their method treats particles as carriers of rotational flow information, whereas the project particles are independent visual elements. The distinction is important: the implemented smoke has plausible drift and expansion, but it does not preserve vorticity or satisfy incompressible flow equations.

Bridson, Hourihan and Nordenstam [10] derive procedural velocity fields using curl noise, enabling incompressible turbulent motion and boundary-aware behavior without a full simulation. This offers a closer extension path for the current application than an adaptive fluid solver. A future version could replace the simple outward smoke velocity with a curl-noise field. The current release does not label its sinusoidal ground pattern or random particle velocities as curl noise.

Kim and colleagues [11] separate large-scale motion from high-frequency turbulence using wavelets. This separation helps artists add detail without recomputing every aspect of the base flow. It also suggests a useful architecture principle: keep mission progression independent from appearance detail. The project follows that separation through distinct Mission and Particles modules, although it does not implement wavelet synthesis.

Cook [3] studies sampling and aliasing in computer graphics and shows how sampling patterns influence visible artifacts. The relevant lesson here is that increased geometric detail alone cannot prevent shimmer. The application requests four-sample MSAA for ordinary window rendering and uses mipmapped minification for its concrete texture. These are distinct practical filters, not an implementation of Cook's stochastic ray-tracing method. Fixed-resolution video capture uses a single-sample target.

Comparative assessment

Adaptive grids [8], hybrid vorticity [9] and wavelet detail [11] offer more physical or multiscale structure than independent billboards. Curl noise [10] offers procedural control with lower integration cost. The current implementation prioritizes understandable data flow and stable interaction, using at most 3,600 particles and two instanced particle draws. The research methods remain specific, testable directions for future improvement.

2.3 Materials and atmospheric appearance

Cook and Torrance [2] describe a reflectance model that accounts for material-dependent angular behavior. Compared with that microfacet approach, Blinn-Phong offers a smaller set of terms that directly matches the course assessment: ambient, diffuse and specular. The project uses the latter and assigns different shininess values to metal-like rocket parts and matte structures. It does not claim physically based material calibration or energy conservation.

Perlin [4] improves the construction of procedural noise, while Lagae and colleagues [15] compare noise families, filtering and their uses in graphics. These works explain why continuity and frequency content matter when procedural patterns are sampled. The implementation uses an original low-frequency trigonometric steppe pattern and a deterministic random concrete image. Neither is described as Perlin noise. More sophisticated noise would improve the natural appearance of the terrain and smoke.

Bruneton and Neyret [12] precompute atmospheric scattering so a renderer can evaluate views from the ground to space efficiently. This approach relates appearance to physical scattering and viewing geometry. The application uses a much simpler blend between a warm horizon and a dark sky as the rocket gains altitude. It meets the visual transition required by the topic, but it cannot reproduce physically correct aerial perspective, planetary shadows or multiple scattering.

Hosek and Wilkie [13] fit an analytic spectral sky model that improves important daylight conditions, including sunsets. Its compact evaluation makes it relevant to real-time graphics, but it targets a more principled radiance distribution than the project's artistic gradient. Hillaire [14] addresses scalable atmosphere rendering with compact lookup-table parameterizations and a multiple-scattering approximation. That method is a stronger future option for continuous ground-to-space views.

Synthesis and source quality

The final design uses simple course-level shading and particles while retaining an architecture that can accept richer material, noise and atmosphere models. The twelve ACM Transactions on Graphics articles and three Computer Graphics Forum articles are journal publications, including journal issues associated with graphics conferences. Their DOI metadata was checked against publisher records. Both venues are Q1 in the 2024 SCImago Computer Graphics and Computer-Aided Design category, using Scopus data as of March 2025 [19]. This is a venue-level classification; the submission checklist records the evidence and ranking convention.

3 Project Description

3.1 Objective

Build a native C++17 application that makes the full graphics pipeline observable through a polished, interactive Baikonur-themed launch sequence. The target is at least 30 FPS at 1280 by 720 on the teaching-assistant machine, with a preference for 60 FPS or more. Results measured on the development machine are reported in Section 6.

3.2 Problem statement

The central problem is coordinating multiple representations in one frame. Rocket parts require consistent parent transforms, smoke requires transparency ordering, lighting requires normals in the correct coordinate space, and the camera must follow an object whose position changes continuously. These systems must remain synchronized when the user pauses, changes playback speed or resets the mission.

3.3 Scope

Area	Implemented behavior
Scene	Steppe, pad, flame trench, gantry, rocket, four boosters, buildings, tanks, radome, roads and lights.
Mission	Eight-second countdown, ignition at T-2, liftoff at T+0, automatic separation at T+24, manual separation after T+5.
Graphics	Indexed meshes, original GLSL, inverse-transpose normals, three light contributions, mipmapped concrete and procedural sky.
Interaction	Keyboard and mouse orbit, zoom, chase mode, pause, reset, speed control, light/night toggles, wireframe and screenshots.
Stretch	Independent booster trajectories, ignition shake, altitude-dependent stars.

The site and vehicle are stylized. Flight dynamics, combustion chemistry, Earth curvature, real telemetry, shadow mapping and collision with the gantry are outside scope. Booster motion uses a visual ballistic arc and a ground-height clamp, not a rigid-body collision solver. The sky transition occurs at compressed demonstration altitudes.

4 Technical Implementation

4.1 Tools and technologies used

The application uses C++17, CMake, GLFW 3.4, the GLAD loader included with GLFW, GLM 1.0.1 and Dear ImGui 1.91.8. Dependencies are included as source and build with the project. Scene shaders use GLSL 330 core. On the test Mac, the driver provides an OpenGL 4.1 core context while the application uses the required 3.3 feature set.

Module	Responsibility
main	Window/context, input, elapsed time, render order, capture and frame logs.
Shader / Mesh / Texture	Compile and link GLSL, own VAO/VBO/EBO resources, upload geometry once, generate the concrete texture and mip chain.
Mission / Camera	Mission state, altitude, booster transforms, orbit/chase viewing and ignition shake.
Scene / Particles	Static scene and rocket hierarchy; particle lifetime updates and instanced billboard passes.
UI / RenderTarget	Controls, telemetry and captions; fixed-resolution recording target displayed in the application window.

4.2 Development process

Development proceeded through a buildable pipeline, primitive geometry, mission transforms, lighting and textures, particles, interaction, visual validation and documentation. Automated tests isolate the simulation from the graphics context. Render captures check behavior that numerical tests cannot establish, including transparency, camera composition and UI placement.

The initial implementation and documents received substantial AI assistance, detailed in Section 9. The package does not claim that the four students independently wrote particular modules or maintained a multiweek commit history. Proposed review and speaking ownership appears in the appendix. Members must record their actual review, changes and test results in genuine commits before submission.

4.3 Algorithms and mathematical model

Transformation pipeline and hierarchy

$$p_{clip} = P\ V\ M_{parent}\ M_{local}\ p_{object}$$

The application uses column vectors. Local scaling shapes each primitive, local translation positions it, and the rocket parent applies ascent. The view matrix comes from the camera eye, target and up vector. A perspective projection uses a 48 degree vertical field of view, the framebuffer aspect ratio, a 0.1 near plane and a 20,000 far plane.

$$n_{world} = normalize\left(transpose\left(inverse\left(mat3\left(M\right)\right)\right)n_{object}\right)$$

The normal matrix preserves perpendicularity under nonuniform scaling. Interpolated normals are normalized again in the fragment shader. Mesh UVs and world positions pass from the vertex stage to fragment evaluation.

Mission and booster motion

$$h(t) = 0.55\ max(t,\ 0)^2; \quad v(t) = 1.1\ max(t,\ 0)$$

The mission clock begins at -8 seconds. At separation time ts, each booster preserves its world position and initial upward velocity. For elapsed time u = t - ts, radial distance grows as 1.72 + 1.8u and vertical position becomes 3.8 + h(ts) + 1.1tsu - 1.8u². Rotation increases at 0.35u radians. Continuity at u = 0 is checked by an automated matrix comparison.

Particle lifecycle and rendering order

Each particle stores position, velocity, age, lifetime, size and material type. Age increases by elapsed time and expired particles are removed. An emission accumulator generates 700 particles per simulation second with a persistent 2:1 fire-to-smoke ratio. The pool is capped at 3,600. Static quad data uploads once; a compact per-instance position/size and color buffer updates each frame.

The frame order is sky, opaque geometry, sorted smoke, additive fire and UI. Smoke sorts back to front using camera distance and blends with source alpha. Fire uses additive blending. Both particle passes keep depth testing against opaque geometry and disable depth writes. A soft radial fragment mask removes the square billboard boundary.

5 Graphics and Rendering

5.1 Scene construction and lighting



Fig. 1 Launch complex with procedural geometry and live telemetry

Figure 1 shows the shared primitive meshes assembled into distinct scene objects. The rocket transform parents the body, nose, bands and boosters. The orange gantry bracing and turquoise vehicle panels establish a consistent palette.



Fig. 2 Night lighting with the toggled gantry point light

Figure 2 shows the scene under a different lighting mode. The renderer combines a directional sun, a blue gantry point light and an orange engine point light. The engine light follows the rocket during ascent.

5.2 Shading and texture sampling

$$I = k_a C + \Sigma_j A_j L_j [C \ max(N \cdot L, 0) + k_s max(N \cdot H, 0)^s]$$

Here C is the surface color, H is the normalized sum of light and view directions, s is shininess, and A is point-light attenuation. Specular contribution is zero when the light lies behind the surface. The shader adds emission for selected lamps and applies a display gamma approximation after shading.

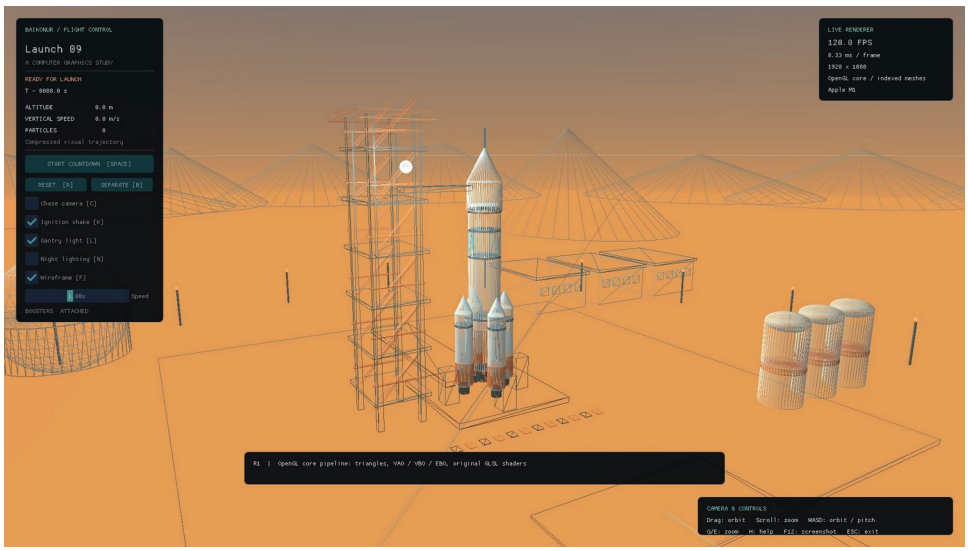


Fig. 3 Wireframe mode exposes indexed triangles in the scene

Figure 3 makes the tessellation and object arrangement visible. All project geometry is submitted through VAOs, vertex buffers and element buffers, with no fixed-function drawing calls.

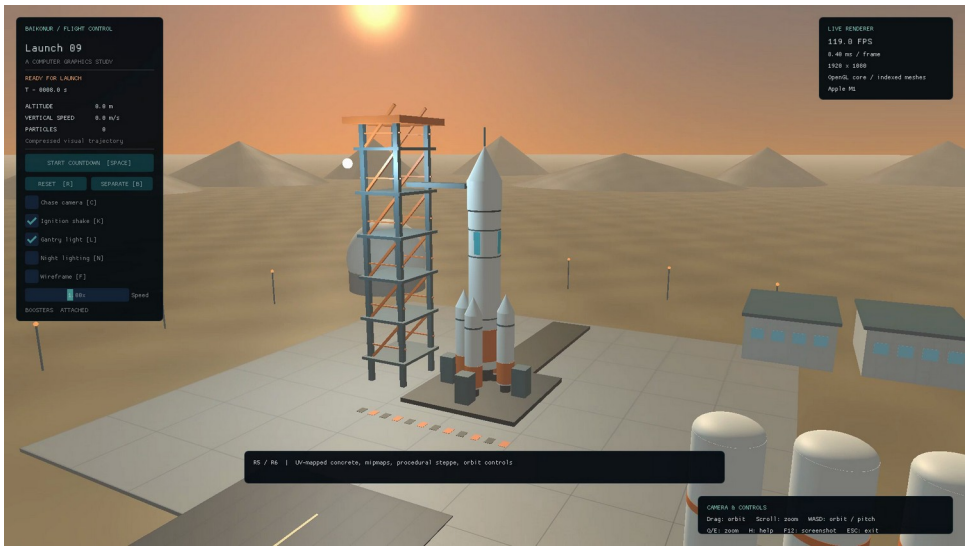


Fig. 4 Repeating concrete UVs and procedural steppe detail

Figure 4 shows the concrete pad with a deterministic 256 by 256 RGB texture. The sampler repeats UVs, uses linear magnification and trilinear mipmapped minification. The steppe uses a separate low-frequency shader pattern. The concrete grid is a stylized material and does not preserve a fixed real-world slab size across differently scaled objects.

5.3 Particle animation and separation

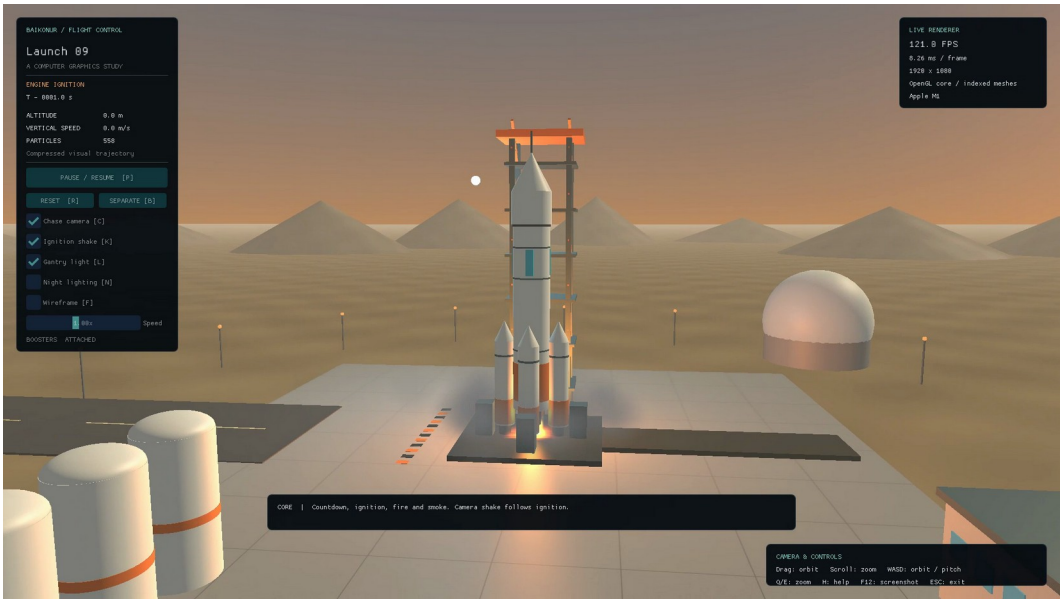


Fig. 5 Ignition combines fire, smoke and a moving engine light

Figure 5 shows warm additive flame and expanding translucent smoke. Ignition also applies a short, decaying camera offset. The K key disables that motion. Pause freezes the mission and particles while allowing independent camera movement.



Fig. 6 Detached boosters against the altitude-dependent star field

Figure 6 shows the topic's final visual event. The shader blends the warm sky toward darkness using clamp(altitude / 450, 0, 1). A direction-based procedural star field fades in at altitude. Booster detachment switches from parent-relative motion to independent world transforms. These are artistic visual effects at compressed altitudes.

6 Testing and Evaluation

6.1 Testing methodology

CTest runs a standalone simulation test executable. It checks grounded idle state, ignition timing, pause, ascent, separation continuity, outward booster motion, equivalent progression at 30 and 60 updates per second, reset, rejection of premature separation, automatic separation, bounded atmosphere values and finite booster transforms. The automated suite passes. Rendering tests compile every shader and check OpenGL errors after each frame. No independent student user study has been conducted.

6.2 Performance evaluation

Hardware: Apple M1 with 8 GiB unified memory, macOS 26.3.1. Compiler: AppleClang 21.0.0. Release build, OpenGL 4.1 Metal driver 90.5. Measurements below use 1280 by 720, four-sample window MSAA, VSync disabled and no recording. Each 12-second run excludes its first two seconds. Mean FPS is sample count divided by the sum of measured frame times; p95 reports the 95th percentile frame time.

Scene	Mean FPS	p95 ms	Max particles
Ready	630.4	4.39	0
Ignition and ascent	757.6	3.36	1399
Separation and stars	1008.8	2.89	1399

These are local frame-loop throughput measurements, including CPU work and buffer swaps. They are not GPU-only timings and do not establish performance on the TA machine. The ready-scene run included a 57.9 ms frame outlier; these averages do not guarantee every frame meets the target. Raw CSV data and exact commands accompany the project. The final demonstration records the actual renderer at 1920 by 1080 with a visible FPS counter and no playback speed change.

Static geometry and the concrete mip chain upload once. Particle rendering uses two instanced draws. The allocated instance buffer is 115,200 bytes (3,600 instances times 32 bytes). The particle pool is bounded; total process/GPU memory was not instrumented, so no unsupported total-memory figure is reported.

6.3 Issues and debugging

Validation found a repeated GLAD implementation include, a high-FPS bias in the fire/smoke mix, overlapping help text, and an incompatible multisample framebuffer blit in video capture. Fixes isolated the loader definition, made the emission ordinal persistent, repositioned help, and used compatible single-sample capture/display targets. Scripted demo input is locked to avoid unrelated user input changing its timeline.

7 Conclusion and Future Work

The project integrates a complete programmable graphics pipeline into an interactive launch scene. It demonstrates hierarchical modeling, per-fragment lighting, UV texture sampling, particle blending, camera control and real-time telemetry. All three Baikonur stretch features are implemented and appear in the demonstration. Automated simulation tests and local rendering checks support the implementation, while the final TA-machine and team defense checks remain separate responsibilities.

7.1 Limitations

The animation follows a visual trajectory rather than a force-based rocket model. The site uses repeated primitives, the terrain is a large flat surface, and the sky is an artistic gradient. Smoke does not collide with the gantry or cast volumetric shadows. The current blend ordering uses particle-center distance, which is an approximation for overlapping billboards. At extreme altitude or camera distance, the finite terrain and long projection range can expose visual limits.

A long application stall is clamped to a 0.1-second simulation update, preventing a sudden visual jump at the cost of falling behind wall time during exceptional delays. MSAA improves ordinary window edges, while the recording target uses single-sample rendering. Performance varies with display synchronization, framebuffer size, capture and background load. The measurements here do not replace a fresh build and run on the actual assessment machine.

7.2 Prioritized future work

The first graphics extension would be directional shadow mapping to anchor the rocket and gantry more convincingly. A second would replace independent smoke drift with a curl-noise velocity field [10] and depth-aware soft particles to improve intersections. A third would introduce a curved ground model and a physical atmosphere approximation based on [12]-[14]. These additions should be profiled individually before increasing visual complexity.

The strongest software improvements would be a more general scene-node representation, finer resource-error cleanup, and platform CI for Windows and Linux in addition to the tested Mac build. A structured user evaluation could measure whether observers can identify the major launch events and operate the controls without assistance. Each improvement should retain the project's central teaching purpose: making the graphics pipeline explainable.

8 References

Journal articles use IEEE-style numbering. DOI records identify the journal versions; conference-associated journal issues are not counted twice.

[1] W. T. Reeves, "Particle Systems - A Technique for Modeling a Class of Fuzzy Objects," ACM Transactions on Graphics, vol. 2, no. 2, pp. 91-108, 1983. https://doi.org/10.1145/357318.357320

[2] R. L. Cook and K. E. Torrance, "A Reflectance Model for Computer Graphics," ACM Transactions on Graphics, vol. 1, no. 1, pp. 7-24, 1982. https://doi.org/10.1145/357290.357293

[3] R. L. Cook, "Stochastic Sampling in Computer Graphics," ACM Transactions on Graphics, vol. 5, no. 1, pp. 51-72, 1986. https://doi.org/10.1145/7529.8927

[4] K. Perlin, "Improving Noise," ACM Transactions on Graphics, vol. 21, no. 3, pp. 681-682, 2002. https://doi.org/10.1145/566654.566636

[5] D. Q. Nguyen, R. Fedkiw, and H. W. Jensen, "Physically Based Modeling and Animation of Fire," ACM Transactions on Graphics, vol. 21, no. 3, pp. 721-728, 2002. https://doi.org/10.1145/566654.566643

[6] A. Lamorlette and N. Foster, "Structural Modeling of Flames for a Production Environment," ACM Transactions on Graphics, vol. 21, no. 3, pp. 729-735, 2002. https://doi.org/10.1145/566654.566644

8 References continued

[7] N. Rasmussen, D. Q. Nguyen, W. Geiger, and R. Fedkiw, "Smoke Simulation for Large Scale Phenomena," ACM Transactions on Graphics, vol. 22, no. 3, pp. 703-707, 2003. https://doi.org/10.1145/882262.882335

[8] F. Losasso, F. Gibou, and R. Fedkiw, "Simulating Water and Smoke with an Octree Data Structure," ACM Transactions on Graphics, vol. 23, no. 3, pp. 457-462, 2004. https://doi.org/10.1145/1015706.1015745

[9] A. Selle, N. Rasmussen, and R. Fedkiw, "A Vortex Particle Method for Smoke, Water and Explosions," ACM Transactions on Graphics, vol. 24, no. 3, pp. 910-914, 2005. https://doi.org/10.1145/1073204.1073282

[10] R. Bridson, J. Hourihan, and M. Nordenstam, "Curl-Noise for Procedural Fluid Flow," ACM Transactions on Graphics, vol. 26, no. 3, Article 46, 2007. https://doi.org/10.1145/1276377.1276435

[11] T. Kim, N. Thurey, D. James, and M. Gross, "Wavelet Turbulence for Fluid Simulation," ACM Transactions on Graphics, vol. 27, no. 3, Article 50, 2008. https://doi.org/10.1145/1360612.1360649

[12] E. Bruneton and F. Neyret, "Precomputed Atmospheric Scattering," Computer Graphics Forum, vol. 27, no. 4, pp. 1079-1086, 2008. https://doi.org/10.1111/j.1467-8659.2008.01245.x

8 References continued

[13] L. Hosek and A. Wilkie, "An Analytic Model for Full Spectral Sky-Dome Radiance," ACM Transactions on Graphics, vol. 31, no. 4, Article 95, 2012. https://doi.org/10.1145/2185520.2185591

[14] S. Hillaire, "A Scalable and Production Ready Sky and Atmosphere Rendering Technique," Computer Graphics Forum, vol. 39, no. 4, pp. 13-22, 2020. https://doi.org/10.1111/cgf.14050

[15] A. Lagae, S. Lefebvre, R. Cook, T. DeRose, G. Drettakis, D. S. Ebert, J. P. Lewis, K. Perlin, and M. Zwicker, "A Survey of Procedural Noise Functions," Computer Graphics Forum, vol. 29, no. 8, pp. 2579-2600, 2010. https://doi.org/10.1111/j.1467-8659.2010.01827.x

[16] Computer Graphics Fundamentals course lecture collection supplied by the team, including Transformations, Viewing, Shading, Texture Mapping, Hierarchical Models and Procedural Methods. Slides credit E. Angel and D. Shreiner, Interactive Computer Graphics, 6th ed., Addison-Wesley, 2012. PDF-page mappings appear in Appendix A and docs/COURSE_MAP.md.

[17] GLFW contributors, GLFW 3.4 documentation, https://www.glfw.org/docs/3.4/ . Accessed October 19, 2026.

[18] Khronos Group, OpenGL 3.3 Core Profile Specification, https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf . API reference for buffers, shaders and drawing.

[19] Matej Bel University Library, Scimago Journal & Country Rank: Computer Graphics and Computer-Aided Design, 2024 metrics (Scopus data as of March 2025), pp. 1 and 3. https://kniznica.umb.sk/app/cmsFile.php?ID=21899&disposition=i .

Library credits and license locations: GLFW, zlib/libpng; GLAD, MIT and Khronos notices; GLM, MIT or Happy Bunny; Dear ImGui, MIT. Exact versions, authors, repository links and retained licenses are listed in CREDITS.md. No external models, texture photographs, sound tracks or copied shader snippets are used.

9 AI Use Note

OpenAI Codex provided substantial assistance in producing this project. It generated the initial C++ application, original procedural geometry and textures, GLSL shaders, simulation tests, build files, documentation, literature-review prose, report and presentation. It also assisted with compiling, debugging, searching and verifying references, measuring performance, and recording the demonstration.

The user selected the Baikonur theme, supplied the assignment documents and course lectures, and provided the four team members' names and groups. At preparation time, the team has not yet confirmed individual source changes, independent verification or actual module contributions. The package therefore does not attribute the generated implementation to particular students or claim a history of student commits.

AI-assisted revisions made during validation included fixing the GLAD definition scope, correcting the emission ratio at high frame rates, moving the controls help to avoid overlap, separating fixed-resolution recording from window size, correcting framebuffer sample compatibility, and locking the scripted demo against unrelated input. These changes were followed by the automated and rendering checks reported in Section 6.

Before submitting, the team must review the implementation, test it using the course setup, confirm or revise the explanations, and record its own substantive changes and responsibilities. Each member must be able to explain the whole pipeline and the mathematics relevant to their presentation. Any further AI-assisted modifications should be added to this note and the README with the specific work performed and how the team checked it.

Asset and source provenance

Geometry, visual textures, shader effects and screenshots originate in the project. The report preserves the university logo from the supplied template. Scholarly papers informed comparisons and design reasoning; their advanced algorithms are not presented as implemented features. The source package retains library license files. The video contains real-time rendered application output with in-application captions and telemetry, encoded without speed alteration.

Appendix A Course connections and defense

Course topic	Project connection
Transforms, L-4-1 pp. 9-10, 22-25	Column vectors, concatenation and local/parent transforms.
Viewing, L-5-1 pp. 11-12, 16-22	LookAt, perspective projection and aspect ratio.
Shading, L-5-2 pp. 29, 66-70	Modified Phong model and per-fragment evaluation.
Textures, L-8 pp. 30-34, 38-40	UV interpolation, repeat sampling and mipmaps.
Hierarchy, L-9 pp. 8-11, 29-32, 45	Parent-child composition and dynamic detachment.
Procedural methods, L-10 pp. 14-22	Particle generation, attributes, motion and extinction.

The references above use PDF page numbers. Course materials supplement the fifteen journal articles rather than replacing them. The detailed mapping also covers geometry, input, solid models and parametric surfaces.

Suggested presentation allocation

Akbope Bakytkeldy (SE2426): slides 1-2, scope and requirements. Moldir Omar (SE2426): slides 3-4, pipeline and hierarchy. Ayaulym Talgat (SE2427): slides 5-7, shading and effects. Talant Altaikhan (SE2425): slides 8-10, interaction, tests and conclusions. This is a proposed speaking plan, not an assertion of completed development contributions.

Submission status

The defense and submission dates remain to be confirmed. The team must verify topic registration, provide its hosted repository, add the TA, maintain genuine member commits, tag the actual submission revision v1.0, and upload the report PDF, slide deck and demo. The submission deadline is three days before defense at 23:59 Asia/Almaty. The local package cannot establish account-based submission or individual oral understanding.